

7TH HEAVEN CAFE — ARCHITECTURE DESIGN

Cloud-Integrated Artisanal Cafe & Bakehouse Digital Ordering & Kitchen Operations System

Project: 7th Heaven Cafe
(Cafe 2)

Platform: Next.js 16, React 19,
TailwindCSS

Backend & DB: Google Firebase &
Firestore NoSQL

Payment & Invoicing: Razorpay API &
Nodemailer

1 FRONT-END DESIGN

Objective: Design and structure what users (customers, kitchen baristas, and administrators) view and interact with, detailing screens, visual screen wireframes, and sequential navigation flowcharts.

A. User Interfaces / Screens

- Home / Brand Landing Page:** Artisanal hero showcase, specialty beverage carousels, cafe philosophy, operating hours, and quick-access ordering CTA.
- Menu & Category Filter Grid:** Filterable by category tabs (*Hot Coffee, Cold Coffee, Refreshers, Milkshakes, Cupcake Milkshakes, Mug Cakes*), dietary toggles (*Vegetarian / Eggless*), and live instant search bar.
- Item Customization Modal:** Allows customers to customize milk preferences (*Oat, Almond, Soy, Regular*), select extra espresso shots, and choose sweetness levels before adding to cart.
- Slide-Over Cart Drawer:** Real-time item listing, unit counter (+/-), addon badges, promotional coupon entry box, live bill breakdown, and checkout button.
- Checkout Screen (/checkout):** Order fulfillment options (*Door Delivery, Table Dine-in, Takeaway Pickup*), customer details, street address with pincode, payment method selection (*Razorpay Online vs Cash on Delivery*), and price breakdown.
- Live Order Tracking Screen (/track-order/[orderId]):** Real-time 4-stage animated stepper (*Placed → Preparing → Out for Delivery → Delivered*), live preparation countdown timer, courier details, and itemized invoice modal.
- Customer Profile & Order History Drawer:** Saved delivery addresses, personal details, past order receipt archive, 1-click reorder button, and secure sign-out.
- Customer Login Screen (/login):** Email and password credentials, Google OAuth One-Tap sign-in, and guest checkout direct continuation.
- Customer Registration Screen (/signup):** Account creation form (Full name, email, 10-digit mobile number, password, and promotional newsletter opt-in).
- Admin / Kitchen Operations Kanban Portal (/admin):** Real-time Kanban board with synchronized order columns (*New Placed, Brewing/Baking, Dispatched, Delivered*), audio chime for incoming orders, and quick status advance controls.
- Menu & Inventory Management Screen:** Real-time item stock availability toggle (`inStock: true/false`), new product creation form, price/addon editor, and catalog delete/archive.
- Sales Analytics & Revenue Dashboard:** Live metrics including Gross Revenue, Completed Ticket Count, Average Order Value (AOV), and top-selling coffees/pastries.
- Coupons & Promotions Management:** Creation and scheduling of promotional codes, minimum subtotal thresholds, percentage or flat discounts, and expiration windows.

B. Screen Design (UI Wireframes)

1. Customer Menu & Slide-Over Cart Interface:

7TH HEAVEN CAFE			[Home]	[Menu]	[About]	[Cart (2)]	[User Icon]
Categories: [All] [Hot Coffee] [Cold Coffee] [Shakes] [Mug Cakes]							
Search: [Search cappuccino, frappe, latte...] [Veg Only]							
[Image: Caramel Latte] Caramel Flan Latte Double shot, velvet milk ₹240 [+ Add to Cart]		[Image: Hazelnut Frappe] Hazelnut Cold Frappe Blended with cream ₹280 [+ Add to Cart]		[Image: Cake] Belgian Mug Warm molten core ₹190 [SOLD OUT]			
CART DRAWER (Slide-over):							
- 1x Caramel Flan Latte (Oat Milk +₹30) ₹270							
- 1x Hazelnut Cold Frappe (Regular) ₹280							
[Coupon Code: 7HFIRST] [APPLY] -> (Success: ₹50 OFF applied!)							
Subtotal: ₹550.00 GST (5%): ₹27.50 Delivery: ₹0.00 Grand Total: ₹527.50							
[PROCEED TO CHECKOUT ->]							

2. Checkout Screen (/checkout):

7TH HEAVEN CAFE – CHECKOUT

[1] FULFILLMENT:

Door Delivery

Dine-in / Table

Takeaway Pickup

[2] CUSTOMER & DELIVERY ADDRESS:

Full Name:

[Aditya Sharma]

Phone Number:

[+91 98765 43210]

Email Address:

[aditya@example.com] (Digital invoice sent here)

Street Address:

[Flat 402, Sunshine Heights, MG Road]

Landmark:

[Near Metro Station Gate 2]

Pincode:

[400001]

[3] PAYMENT METHOD:

Pay Online via Razorpay (UPI, Credit/Debit Card, Netbanking)

Cash on Delivery (COD)

[4] BILL BREAKDOWN:

Caramel Flan Latte (x1) ₹270 | Hazelnut Frappe (x1) ₹280

Item Subtotal: ₹550.00 | GST (5%): ₹27.50 | Discount: -₹50.00 | Delivery: FREE

NET AMOUNT PAYABLE: ₹527.50

[PLACE ORDER & PAY ₹527.50]

3. Real-Time Order Tracking Screen (/track-order/[orderId]):

ORDER TRACKING: #7H-20260303-0042

Status: KITCHEN IS CURRENTLY PREPARING YOUR ORDER

Estimated Delivery: ~18 Minutes (Target Arrival: 09:15 AM)

[(✓) Placed]

08:45 AM

[(●) Preparing]

At Espresso Station

[() On the Way]

With Delivery Agent

[() Delivered]

Pending

Items Ordered: 1x Caramel Flan Latte (Oat Milk), 1x Hazelnut Cold Frappe

Delivery Destination: Flat 402, Sunshine Heights, MG Road

Payment Status: ₹527.50 (Paid Online via Razorpay #pay_823746)

[View / Download Tax Invoice]

[Contact Cafe Support]

4. Admin & Kitchen Real-Time Kanban Board (/admin):

7TH HEAVEN KITCHEN DISPATCH

Sound: ON

Active Orders: 6

Auto-Sync

[NEW PLACED (2)]

[PREPARING (2)]

[DISPATCHED (1)]

[DELIVERED]

Order #7H-0043 (COD)

Time: 2 mins ago

- 2x Belgian Mug Cake

Total: ₹480

[ACCEPT & PREPARE ->]

Order #7H-0042 (PAID)

Time: 11 mins ago

- 1x Caramel Latte

Total: ₹527.50

[DISPATCH ORDER ->]

Order #7H-0040

Courier: Sunil K.

ETA: 6 mins

[Mark Delivered]

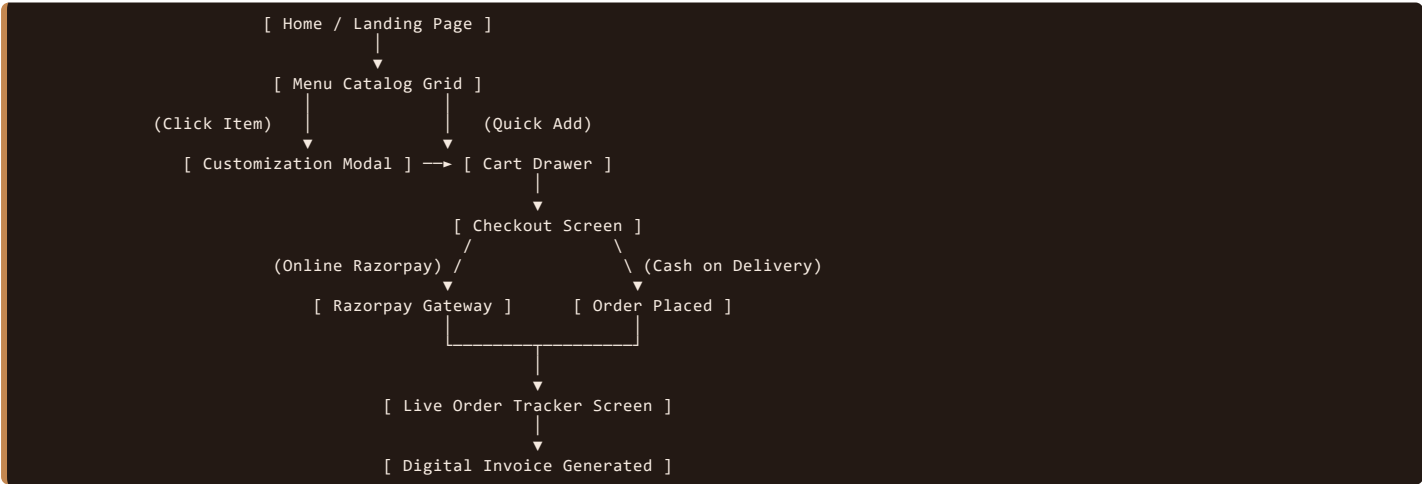
Order #0039

Completed

₹680 (COD)

[Archived]

C. Navigation Flow



2 BUSINESS LOGIC DESIGN

Objective: Business Logic explains how the system processes input, applies pricing formulas, executes security checks, and manages state machines across the entire order lifecycle.

A. Identify System Modules

- **Authentication & User Session Module:** Firebase Auth integration, JWT verification, role-based access control (`customer`, `kitchen`, `delivery`, `admin`), and guest checkout session persistence.
- **Catalog & Real-time Stock Module:** Item category filtering, live stock toggle (`inStock: true/false`), and dynamic price retrieval.
- **Cart & Server-Side Pricing Engine:** Independent recalculation of line items, 5% GST tax, delivery fee waivers, and promo discounts.
- **Payment Gateway Module:** Server-side Razorpay order generation (`/api/razorpay`) and cryptographic HMAC-SHA256 signature verification.
- **Order Lifecycle & Kitchen Queue Module:** State progression machine (`placed` → `preparing` → `out_for_delivery` → `delivered`) using real-time Firestore listeners (`onSnapshot`).
- **Automated Invoicing & Notification Module:** Automated dispatch of itemized HTML invoices via Nodemailer upon successful order placement.

B. Business Rules & Order Placement Logic

Key Operational Rules: 1. Cart must contain ≥ 1 item | 2. All items must have `inStock == true` in Firestore at checkout | 3. Prices are recalculated server-side | 4. Subtotal $\geq ₹500$ waives delivery (₹0), else ₹40 | 5. 5% GST applied | 6. Razorpay signatures verified cryptographically.

Step-by-Step Processing Logic:

```
Customer clicks [ PLACE ORDER ]
↓
Validate customer input fields (Name, Phone, Address, Email)
Fetch current MenuItem documents from Firestore by ID
IF any item.inStock == false THEN Display Error: "Item [Item Name] is sold out." -> HALT ENDIF
Calculate line total = (Base Price + Sum of Addons) * Quantity; Subtotal = Sum of all Line Totals
IF Subtotal >= ₹500 THEN Delivery Fee = ₹0 ELSE Delivery Fee = ₹40 ENDIF
Apply verified coupon discount (if valid)
Taxes = Subtotal * 0.05 (5% GST); Grand Total = Subtotal + Taxes + Delivery Fee - Discount
↓
IF Payment Method == "Online (Razorpay)" THEN
  Create Razorpay Order (amount in paise = Grand Total * 100) -> Prompt checkout modal
  Verify returned signature: HMAC_SHA256(order_id + '|' + payment_id, secret)
  IF signature mismatch THEN Record status = "failed" -> Display Error -> HALT ENDIF
  Set paymentStatus = "paid"
ELSE
  Set paymentStatus = "pending" (Cash on Delivery)
ENDIF
↓
Atomically increment counter -> Generate Order No: "7H-YYYYMMDD-XXXX"
Create document in orders collection with status: "placed"
Trigger background Nodemailer invoice to customer email
Broadcast real-time ticket to Kitchen Kanban display -> Redirect to `/track-order/[orderId]`
```

C. Kitchen Queue & Order Fulfillment Logic

```
Order created in Firestore with status: 'placed'
↓
Kitchen staff views ticket on Kanban board & clicks [ Accept & Prepare ]
Update status = 'preparing', estimatedDeliveryAt = Current Time + 25 Minutes
Firestore onSnapshot pushes instant update to customer's live tracking view
↓
Chef finishes brewing & packaging; clicks [ Dispatch Order ]
Update status = 'out_for_delivery' -> Customer tracking screen shows courier on route
↓
Delivery agent hands package to customer; clicks [ Confirm Delivered ]
Update status = 'delivered', deliveredAt = Current Timestamp
IF paymentMethod == "Cash on Delivery" THEN paymentStatus = 'paid' ENDIF
Trigger post-fulfillment star rating & customer review prompt
```

D. Pricing & Tax Calculation Formulas

1. **Line Item Total:** $\text{\text{\$}}\{\text{Item Line Total}\} = (\text{\text{\$}}\{\text{Base Price}\} + \sum \text{\text{\$}}\{\text{Selected Addon Prices}\}) \times \text{\text{\$}}\{\text{Quantity}\}$
2. **Taxes (GST):** $\text{\text{\$}}\{\text{GST Amount}\} = \text{\text{\$}}\{\text{Subtotal}\} \times 0.05 \quad (5\%)$
3. **Delivery Fee:** $\text{\text{\$}}\{\text{Delivery Fee}\} = \text{\text{\$}}\{\text{if Subtotal} \geq \text{\text{\$}}500 \text{ then } \text{\text{\$}}0 \text{ else } \text{\text{\$}}40\}$
4. **Coupon Discount:** $\text{\text{\$}}\{\text{Discount}\} = \min(\left(\text{\text{\$}}\{\text{Subtotal}\} \times \frac{\text{\text{\$}}\{\text{Discount Percentage}\}}{100}\right), \text{\text{\$}}\{\text{Max Discount Cap}\})$

Complete Numerical Calculation Example:

- 1x Caramel Flan Latte (Base ₹240 + Oat Milk ₹30) = ₹270.00 | 1x Hazelnut Cold Frappe (Base ₹280) = ₹280.00
- Subtotal:** ₹270 + ₹280 = **₹550.00**
- Delivery Fee:** ₹0.00 (Since ₹550 ≥ ₹500 threshold)
- Coupon Applied:** 7HFIRST (10% off, max ₹60) → Discount = $\min(550 \times 0.10, 60) = \text{\text{\$}} \textbf{₹55.00}$
- GST (5%):** $₹550 \times 0.05 = \text{\text{\$}} \textbf{₹27.50}$
- Grand Total:** $₹550.00 + ₹27.50 + ₹0.00 - ₹55.00 = \text{\text{\$}} \textbf{₹522.50}$

E. Validation Rules

INPUT FIELD	VALIDATION RULE / CONSTRAINT	ERROR HANDLING OUTPUT
Customer Name	Required, alphabetic & spaces, 2 to 60 characters	"Please enter a valid full name."
Email Address	Required, valid RFC 5322 email regular expression	"Please enter a valid email for invoice receipt."
Mobile Number	Required, exactly 10 digits starting with 6-9	"Enter a valid 10-digit mobile number."
Delivery Address	Required for delivery orders, minimum 10 characters	"Complete delivery address is required."
Cart Quantity	Integer ≥ 1 and ≤ 20 per item	"Quantity must be between 1 and 20."
Item Stock	Must verify <code>inStock == true</code> in Firestore	"Item is currently sold out."
Coupon Code	Active, non-expired, and <code>Subtotal ≥ minOrderValue</code>	"Coupon invalid or order threshold not met."
Payment Signature	HMAC-SHA256 hash match with Razorpay secret key	"Transaction security verification failed."

F. User Roles and Permissions

USER ROLE	ALLOWED OPERATIONS & ACCESS CONTROLS
Guest User	Browse menu catalog, customize beverages, add to cart, execute checkout, track order by URL ID.
Customer (Registered)	All Guest features, plus: save multiple delivery addresses, view order history, 1-click reorder, rate orders.
Kitchen Staff / Barista	Access <code>/admin</code> kitchen queue, receive live order chimes, advance status (Placed → Preparing), toggle item stock status.
Delivery Rider	Access assigned deliveries, view customer contact & address, advance status (Dispatched → Delivered), mark COD collected.
Cafe Administrator	Full platform access: add/update/delete menu catalog items, manage promotional coupons, view daily/monthly sales analytics, process refunds.

3 DATABASE DESIGN

Objective: Detail how data is structured and stored using Google Cloud Firestore (NoSQL), including document collections, frozen point-in-time snapshot models, primary keys, and foreign references.

A. Identify Entities / Collections

1. `users` | 2. `menu_items` | 3. `orders` | 4. `coupons` | 5. `order_sequences`

B. Table / Collection Structures

1. Collection: `users`

FIELD NAME	DATA TYPE	KEY	DESCRIPTION & CONSTRAINTS
<code>uid</code>	String	PK	Firebase Auth UID (Unique identifier for customer/staff)
<code>name</code> , <code>email</code>	String	Unique	Required full name and unique email address for login & invoices
<code>phone</code> , <code>role</code>	String		Contact phone; Enum: ['customer', 'kitchen', 'delivery', 'admin']
<code>addresses</code>	Array of Objects		List of saved addresses: [{ tag, street, landmark, city, pincode, isDefault }]
<code>createdAt</code>	Timestamp		Account creation timestamp

2. Collection: `menu_items`

FIELD NAME	DATA TYPE	KEY	DESCRIPTION & CONSTRAINTS
<code>id</code>	String	PK	Unique auto-generated document ID
<code>name</code> , <code>category</code>	String		Required name; Enum: ['Hot Coffee', 'Cold Coffee', 'Refreshers', 'Milkshakes', 'Mug Cakes']
<code>price</code> , <code>inStock</code>	Number, Boolean		Base price in INR (₹); Default <code>inStock</code> : true (Real-time availability toggle)
<code>isVegetarian</code> , <code>addons</code>	Boolean, Array		Dietary indicator; Array of customizable options: [{ name: 'Oat Milk', price: 30 }]
<code>image</code> , <code>description</code>	String		Cloud Storage media URL and flavor profile description

3. Collection: `orders`

FIELD NAME	DATA TYPE	KEY	DESCRIPTION & CONSTRAINTS
<code>id</code> , <code>orderNumber</code>	String	PK	Auto document ID; Unique readable invoice number (e.g., '7H-20260303-0042')
<code>userId</code>	String	FK	References <code>users.uid</code> or 'guest' for non-registered checkouts
<code>customerName</code> , <code>email</code> , <code>phone</code>	String		Contact details for billing and delivery coordination
<code>customerAddress</code> , <code>orderType</code>	String		Street address with landmark; Enum: ['delivery', 'takeaway', 'dine_in']
<code>items</code>	Array of Objects		Frozen Snapshot: [{ menuItemId, name, price, qty, selectedAddons, itemTotal }]
<code>subtotal</code> , <code>taxes</code> , <code>deliveryFee</code>	Number		Line item subtotal; 5% GST; Shipping fee (₹0 or ₹40)
<code>discount</code> , <code>grandTotal</code>	Number		Coupon deduction amount; Final net charge amount
<code>paymentMethod</code> , <code>paymentStatus</code>	String		'Online (Razorpay)' or 'Cash on Delivery'; 'pending' / 'paid'
<code>status</code>	String		Enum: ['placed', 'preparing', 'out_for_delivery', 'delivered', 'cancelled']
<code>placedAt</code> , <code>estimatedDeliveryAt</code>	Timestamp		Order submission time and target ETA for customer countdown

4. Collection: `coupons`

FIELD NAME	DATA TYPE	KEY	DESCRIPTION & CONSTRAINTS
<code>code</code>	String	PK	Uppercase alphanumeric voucher code (e.g., '7HFIRST')
<code>discountType</code> , <code>discountValue</code>	String, Number		'percentage' or 'flat'; Discount quantum
<code>minOrderValue</code> , <code>maxDiscount</code>	Number		Minimum cart subtotal; Maximum rupee deduction cap
<code>validUntil</code> , <code>isActive</code>	Timestamp, Bool		Expiration timestamp; Toggle to enable/disable code

Primary Key & Foreign Key Mapping & Snapshot Pattern:

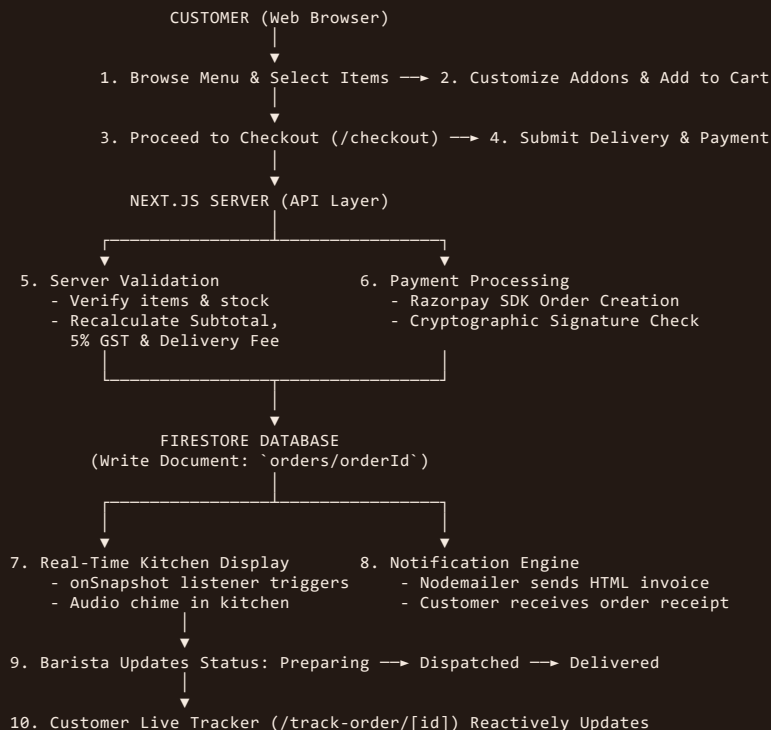
- `users.uid`, `menu_items.id`, `orders.id`, `coupons.code` act as Primary Keys.
- `orders.userId` → references `users.uid` (FK relation).
- Frozen Snapshot Pattern:** `orders.items` embeds exact point-in-time names, unit prices, and addons. Any subsequent price edits in `menu_items` will never mutate past financial invoices.

4 ER DIAGRAM (Entity Relationship Diagram)

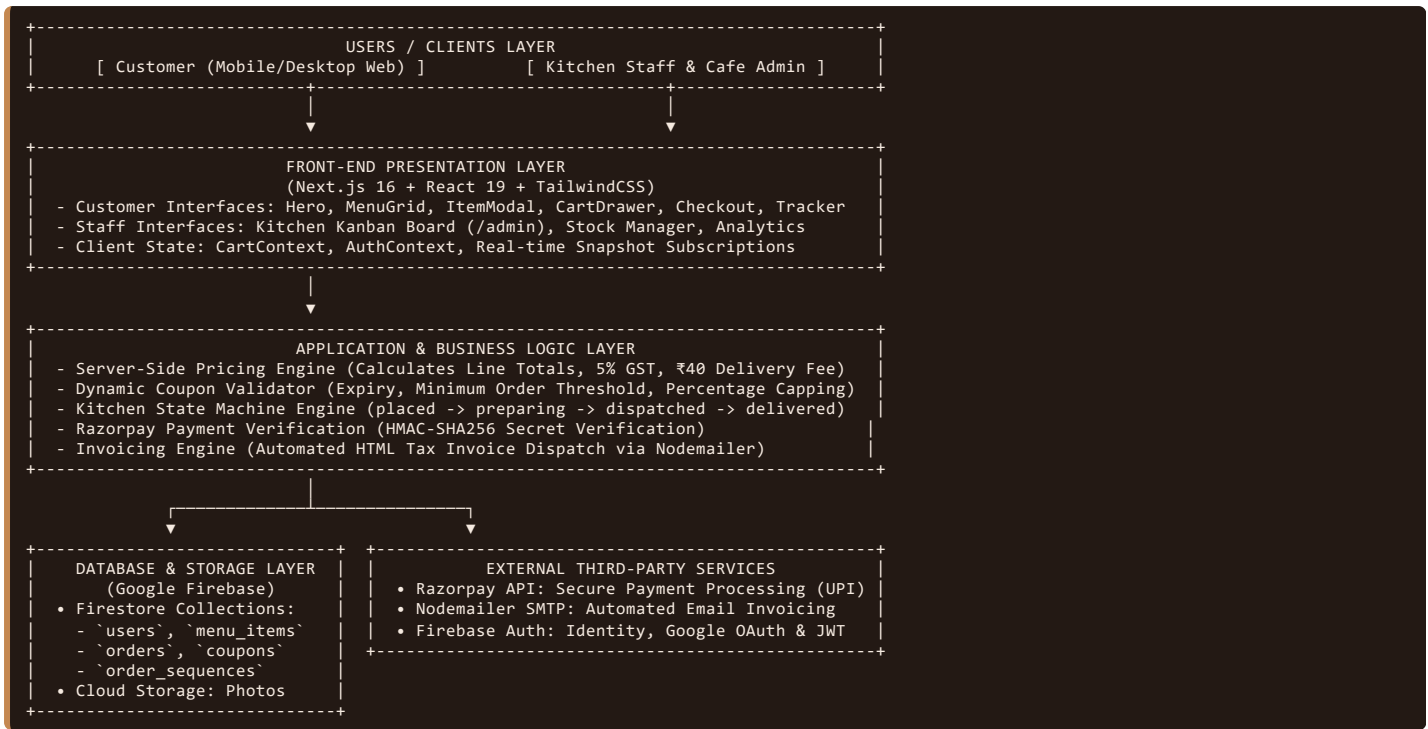


Cardinality: USER → ORDER (1 : M); MENU_ITEM → ORDER (1 : M via snapshots); COUPON → ORDER (1 : M).

5 DATA FLOW / SYSTEM FLOW



6 COMPLETE ARCHITECTURE OVERVIEW



7 FINAL CHECK: COMPLETE "PLACE ORDER" FEATURE WALKTHROUGH

ARCHITECTURE LAYER	FEATURE EXECUTION FOR 7TH HEAVEN CAFE
What does the user see? (Front-End Design)	- Customer selects a <i>Caramel Flan Latte</i> and <i>Belgian Mug Cake</i> from <code>MenuGrid</code>. - Selects "Oat Milk" in the <code>ItemModal</code> and clicks "Add to Cart". - Opens <code>CartDrawer</code>, views bill summary with GST, and applies promo coupon <code>7HFIRST</code>. - Clicks "Proceed to Checkout", fills delivery address, selects "Pay Online via Razorpay", and clicks "Place Order". - Razorpay popup opens; upon payment completion, the screen automatically routes to <code>/track-order/[orderId]</code> showing the animated 4-stage preparation stepper.
What happens on action? (Business Logic)	- API independently verifies stock availability for both items in Firestore. - Computes line totals and applies 10% coupon deduction. - Adds 5% GST and sets delivery fee to ₹0 (since subtotal ≥ ₹500). - Creates Razorpay order in paise; on frontend return, validates cryptographic payment signature. - Generates human-readable sequential ticket number (<code>7H-20260303-0042</code>). - Sends itemized digital invoice with order number to customer's email via Nodemailer. - Alerts kitchen workstation with an audio chime and puts order ticket in the "New Placed" Kanban queue.
Where is it stored? (Database Design)	- A new document is written to the <code>orders</code> collection containing frozen snapshot items, totals, delivery address, and initial status <code>'placed'</code>. - The <code>order_sequences</code> counter document is atomically incremented by +1. - The customer's saved address is optionally updated in their document under the <code>users</code> collection. - When baristas advance tickets on <code>/admin</code>, the <code>status</code> attribute in <code>orders</code> changes to <code>'preparing'</code>, which automatically updates the customer's live tracking screen via Firestore's <code>onSnapshot</code> channel.